Team Number 1

Incident Report Assistant — Final Report

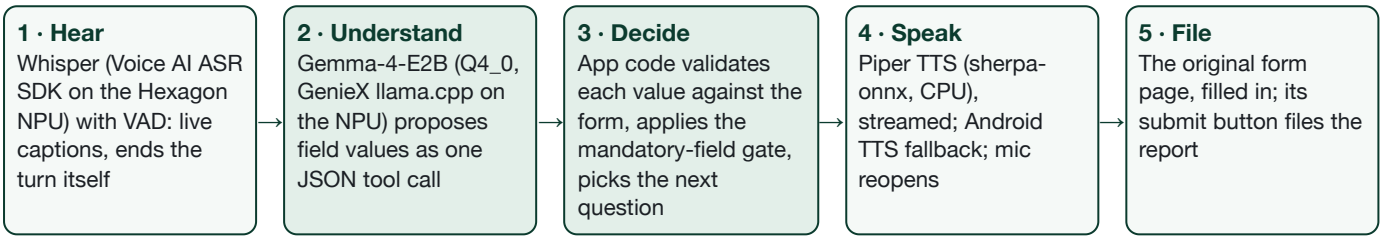
Qualcomm Problem Statement 1 · runs entirely on the QIDK (Snapdragon 8 Gen 3, SM8650, Hexagon v75) · Android

1. The problem and our answer

Incidents (a leak, a fall, a fire alarm, a security breach) are often reported late or incompletely because filling a form on a phone is slow, and the person who saw it is busy. The task: let them **just say what happened**, have the device fill the organisation's incident form, **ask only for what is missing**, and do it all on the device.

Our app is a voice assistant for **any incident form**. The worker describes the incident in their own words; the assistant fills every field it can, asks short follow-up questions until all mandatory fields are complete, and ends on **the organisation's own form page, already filled in**, ready to submit. Speech recognition, the language model and speech synthesis all run on the QIDK — no cloud, no network needed.

2. Solution approach



Four design decisions

1. **The model proposes, the code decides.** Gemma never writes to the form directly. It may only call `record_fields` (or `off_topic`), giving each field a value, a confidence and the user's own words as evidence; `ask_user` and `submit_report` calls are ignored — the app's dialog policy owns the questions. Deterministic Kotlin checks field ids, types and options, merges corrections, asks back when unsure, and enforces the mandatory-field gate. The model cannot submit; the gate is re-checked at submit, and each draft has one report id, so a report is never filed twice. Simple answers (“medium”, “yes”) take a fast path that skips the model.
2. **The form is data, read from ordinary HTML.** The app is given an HTML form page (a URL in production, a pushed or bundled file here). It reads only standard HTML semantics — `<label>`, `required`, `<select>`/radio/checkbox options, input types (date, time, number, email, tel …), `<fieldset>`, `title`/`placeholder`. Nothing about any particular form is written in the app; the page's own scripts are ignored. A new form needs no code change and no rebuild.
3. **The model reads each new form once.** When a form loads, Gemma writes a natural spoken question for every field and identifies which fields are the reporter's own name, email and phone. The reading is cached on the device, keyed by the form's fields, so it costs nothing on later launches and is redone only when the form changes.
4. **Three models share one phone — we orchestrate them.** On the QIDK, Whisper and Gemma share the Hexagon DSP: Whisper must initialise first (it puts the DSP library path in place for Gemma's NPU backend), and the two must never run inference at the same time — we reproduced an out-of-memory crash when they did. The app serialises them: the mic is usable as soon as Whisper is ready, Gemma's work waits while a recording is being transcribed, and a tap during Gemma's work starts listening the moment it finishes. TTS runs on the CPU, so it never competes for the NPU.

3. Features

Reporting by voice

- Free speech in, structured report out: one sentence can fill several fields (type, place, time, severity, actions).
- Follow-ups only for missing mandatory fields; optional ones grouped into one question and asked once.
- “I don't know” → marked “not specified” after confirmation; corrections (“actually it was 11”) overwrite.
- Off-topic speech is politely redirected without touching the draft.

Forms, review and submission

- Any HTML incident form — plain, styled, or with JavaScript; swap it with one command (`push-form.sh`).
- The reporter is never asked: name/email/phone come from a one-time profile and go into whatever fields the form uses.
- Review screen with the right editor per field: date/time pickers, choice chips, tick boxes, numeric/phone/email keyboards.

- Hands-free loop: voice-activity detection ends each turn, the reply is spoken, then the mic reopens; typing always works.
- One-tap home-screen widget: opens the assistant and starts recording immediately.
- Relative times (“yesterday around 4 pm”) are resolved against the device clock and stored in the field's own format.
- The final step is the organisation's own form page, filled in; edits made on it are kept; its submit files the report.
- Report JSON out: values, confidence, evidence, transcript and turn metrics; saved on the device, optional POST to a server.
- TTS is complementary: it never blocks the on-screen flow. Minimal UI: three model status dots and a profile menu.

What sets this solution apart

1. **Form-agnostic for real.** It works from the HTML form an organisation already has — not a schema written for the app. Labels, mandatory fields, options and input types are read from standard HTML; plain, styled and scripted pages all work (the page's scripts are ignored). Four different incident forms run with zero code changes, and a new one is swapped in with one command, without rebuilding the app.
2. **The LLM is never trusted to act.** It only proposes values; validation, the mandatory-field gate and submission are deterministic code. Hallucinated fields or options are rejected, the model's own “submit” is ignored, and nothing is filed without the user's confirmation — with one report id per draft, so a report is never filed twice.
3. **Every value is backed by evidence.** Each field keeps the user's own words and a confidence. Unsure values are read back for confirmation, “I don't know” becomes “not specified” only after the user agrees, and corrections overwrite cleanly.
4. **Questions are written for each form, not for the app.** When a new form arrives, the on-device model writes a natural spoken question for every field, and the reading is cached, so it costs nothing on later launches.
5. **The reporter is never asked who they are.** A one-time profile fills the form's own name, email and phone fields — found by the model even when the form does not mark them — so nobody has to answer “reported by: me”.
6. **The loop closes on the organisation's own form.** The user ends on exactly the page they would have typed into, filled in; edits made there are kept and its own submit button files the report, so it plugs into the existing process.
7. **Three models on one Snapdragon, two on the NPU — orchestrated, not just bundled.** We measured the start-up order the shared Hexagon DSP needs and serialise Whisper and Gemma so they never collide; the mic is ready seconds after launch. Nothing ever leaves the device.
8. **Fast where it can be.** Common answers (“medium”, “yes”) are understood in about 2 ms without calling the model, and speech starts playing within about 220 ms of a reply.
9. **Usable in any situation.** Voice-only and hands-free, or by typing; spoken prompts step aside when a screen reader is active; the review screen gives each field the right editor (date and time pickers, choices, tick boxes, phone and email keyboards).

4. Results (measured on the QIDK)

What	Result	Notes
Gemma-4-E2B load on the NPU	median 3.5 s (2.9–4.7 s, n = 24)	Gemma 4 E2B, Q4_0, on the Hexagon NPU through GenieX.
Whisper ready after app start	≈ 1.2 s	Voice AI ASR SDK, Whisper-Small on the HTP.
Piper TTS: load / first audio	0.73 s / median 220 ms (max 416 ms, n = 17)	Synthesis keeps pace with playback (real-time factor 0.82–1.05), streamed as it is generated.
Mic ready after launch	4.9 s (was ≈ 15 s)	Form reading cached and models serialised; the form is ready 1.3 s after launch.
Reading a new form (once per form)	≈ 6–10 s	One Gemma call; then cached on the device.
Answer → next question, with the model	median 6.4 s (2.0–9.8 s, n = 13)	Dominated by Gemma generation time; grows with how much was said.
Answer → next question, fast path	≈ 2 ms (n = 2)	Option words like “medium” or “yes” need no model call.

What	Result	Notes
Valid tool call from the model	13 of 14 model turns	The one failure produced no tool call; the draft was left unchanged and the assistant asked its next question.
End-to-end on the device	report filed	Typed report → follow-ups → review → filled HTML form → submit (reference RPT-91F8E918).
Automated tests (host)	168 / 168 pass	Session logic, extraction parsing, HTML parsing, filled-form rendering, date formats, every bundled form.

5. Models and resources

Model / library	Purpose	Runs on	License
Whisper (quantized, + VAD)	Speech to text	Hexagon NPU · Voice AI ASR SDK	Qualcomm (QPM)
Gemma-4-E2B-it, Q4_0 GGUF	Extraction + tool calling, form reading	Hexagon NPU · GenieX 0.7.1	Apache-2.0
Piper en_US lessac-medium (VITS)	Speech synthesis	CPU · sherpa-onnx 1.13.8	MIT
jsoup 1.18.1	HTML form → form schema	App (Kotlin)	MIT
Jetpack Compose, Material 3, kotlinx-serialization	UI, widget, JSON	App (Kotlin)	Apache-2.0

Code: Kotlin, Android (minSdk 27). Setup: `fetch-whisper-sdk.sh`, `fetch-piper-tts.sh`, `push-whisper-models.sh`, `push-gemma-model.sh`, then `./gradlew :app:assembleDebug`; a form is swapped with `push-form.sh`. Project licensed MIT.